សប្តាហ៍ទី 3 · Records និងទិន្នន័យ

Programming for Information Systems

record មួយ សម្រាប់សំណើនីមួយៗ · ID មួយ សម្រាប់ record នីមួយៗ

ចាប់ផ្តើមថ្នាក់ · ចែករំលែកកិច្ចការស្រាវជ្រាវទី 2

ជាក្រុមបីនាក់ ចែករំលែកវិធានដែលអ្នកបានស្រាវជ្រាវ។

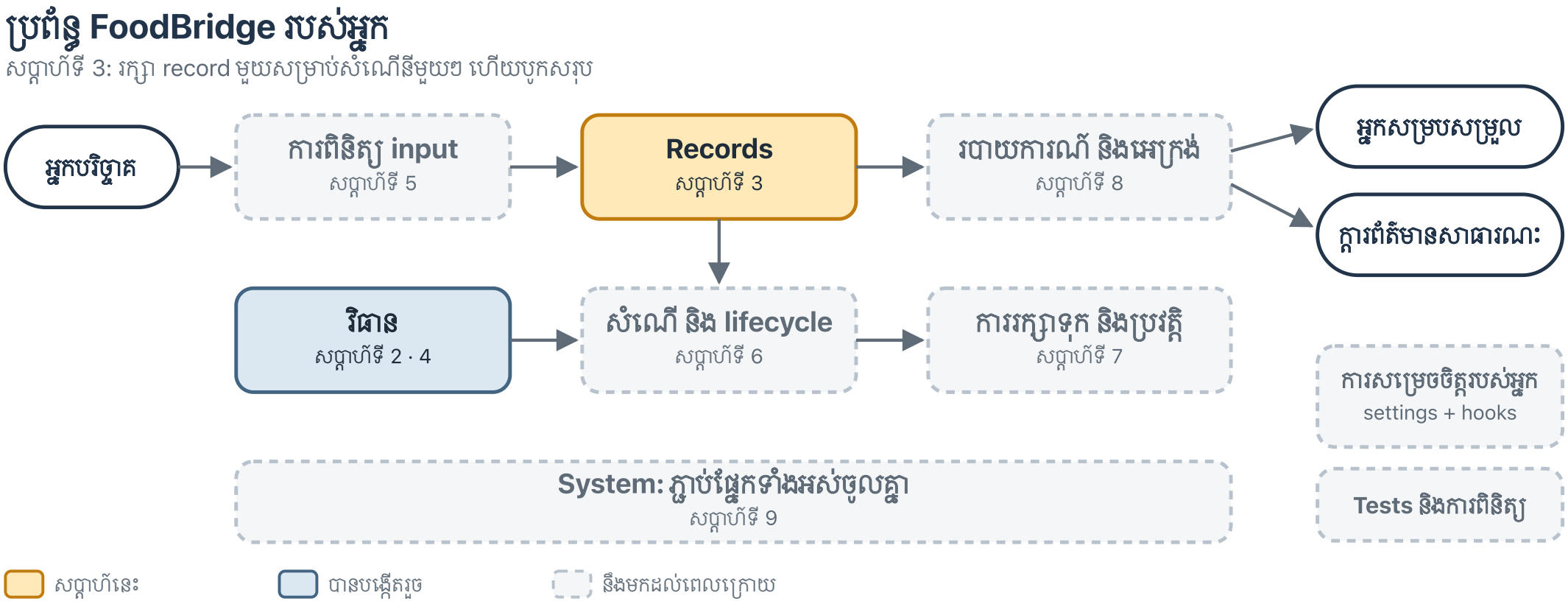
1. អានវិធានរបស់អ្នកឱ្យឮៗ ឱ្យ **ត្រឹមត្រូវ**។ តើក្រុមអាចរកឃើញករណីដែលវាមិនគ្របដណ្តប់ទេ?
2. ចែករំលែកករណី **boundary** របស់អ្នក។ តើគ្រប់គ្នាយល់ស្របទេ ថាតើតម្លៃឡើង ត្រង់ចំណុចកំណត់តែម្តង?
3. ចែករំលែកករណី **ចម្លែក** (បាត់ អវិជ្ជមាន មិនធម្មតា)។ តើប្រព័ន្ធគួរធ្វើអ្វី?

> 📝 កត់ត្រា edge case មួយ ពីមិត្តរួមថ្នាក់។

សប្តាហ៍នេះ

សំណួរ	តើប្រព័ន្ធមួយចង់ចាំរបស់ច្រើន ហើយបែងចែកវាពីគ្នា ដោយរបៀបណា?
Python	list, dictionary, list នៃ dictionary, `for`, accumulator, ការស្វែងរកតាម ID
គោលការណ៍ទី 3	record មួយ សម្រាប់របស់ពិតមួយ: record មួយ ID មួយមិនស្ទួន គ្មានព័ត៌មានណារក្សាទុកពីរដង
អ្នកបង្កើត	`total_portions`, `find_offer` (⭐⭐ `portions_by_category`)

ប្រព័ន្ធរបស់អ្នក ត្រឹមសប្តាហ៍ទី 10



គំនិតសំខាន់: record និង ID

គិតអំពី **បញ្ជីឈ្មោះនិស្សិត** ក្នុងថ្នាក់។

- និស្សិតម្នាក់ៗ គឺជា **ជួរដេកមួយ**។ គ្រប់ជួរដេកមានជួរឈរដូចគ្នា។
- និស្សិតពីរនាក់អាចមានឈ្មោះដូចគ្នា ប៉ុន្តែ **មិនដែល** មាន ID និស្សិតដូចគ្នាទេ។
- ការិយាល័យរកអ្នកតាម **ID** មិនមែនតាមឈ្មោះទេ។

> ក្នុងប្រព័ន្ធរបស់យើង សំណើនីមួយៗគឺជា **record** មួយ ដែលមាន **field** ដូចគ្នា និង **ID** មួយមិនស្ទួន ("F001")។ បញ្ជី field ទាំងនោះ គឺជា **data model**។

data model: តើ field ណាខ្លះគួរមាន?

Field	ឧទាហរណ៍	ហេតុអ្វីយើងរក្សាវា
`id`	`"F001"`	ចែងចែកសំណើពីគ្នា ទោះអាហារដូចគ្នាក៏ដោយ
`donor`, `contact`	`"Sunrise Bakery"`, `"sunrise@example.org"`	អរគុណ និងទាក់ទងទៅវិញ (contact ជាព័ត៌មានឯកជន)
`food`, `category`	`"Bread rolls"`, `"bread"`	វាជាអាហារអ្វី; category សម្រេចថាទទួលយកឬអត់
`portions`, `hours_left`	`30`, `5`	មានប៉ុន្មាន និងត្រូវទៅយកឆាប់ប៉ុណ្ណា
`status`, `partner`	`"approved"`, `"Hope Children's Centre"`	វានៅដំណាក់កាលណា អ្នកណាជាអ្នកទទួល

- **មិន** នៅក្នុង record: "ចំណែកសរុបថ្ងៃនេះ"។ ផលបូកត្រូវ **គណនា** ពេលត្រូវការ។
- ហេតុអ្វីប្រើ ID មិនប្រើឈ្មោះអាហារ? ឈ្មោះអាហារអាចដូចគ្នា ហើយអាចត្រូវគេកែ។ ID មិនដែលដូច្នេះទេ។

មូលដ្ឋាន Python

list, dictionary និង loop

list រក្សា value តាមលំដាប់

```python
volunteers = ["Dara", "Sophea", "Vanna"]
print(volunteers[0])          # Dara: ទីតាំងចាប់ផ្តើមពី 0
print(len(volunteers))        # 3
volunteers.append("Kosal")    # បន្ថែមមួយនៅខាងចុង
print("Sophea" in volunteers) # True
# print(volunteers[4])        # IndexError
```

- លេខទីតាំង គឺជា **index**។ index ទីមួយគឺ 0។

dictionary ដាក់ឈ្មោះឱ្យ value នីមួយៗ

```
item = {"name": "Notebook", "price_riel": 4000, "in_stock": 25}
print(item["price_riel"])            # 4000
item["price_riel"] = 3500            # ប្តូរ value មួយ
item["colour"] = "blue"              # បន្ថែម key ថ្មីមួយ
print(item.get("size"))              # None, គ្មាន error
print(item.get("size", "one size"))  # default ដែលអ្នកជ្រើសរើស
```

- គូនីមួយៗគឺ **key: value**។ រកតាម key មិនមែនតាមទីតាំងទេ។
- `item["size"]` នឹងឈប់ដោយ `KeyError`; `.get` សួរដោយគួរសម។

record គឺជា dictionary; មួយថ្ងៃគឺជា list

```
offers = [
    {"id": "F001", "food": "Bread rolls", "portions": 30},
    {"id": "F002", "food": "Fried rice boxes", "portions": 18},
]
print(len(offers))           # 2
print(offers[1]["food"])     # Fried rice boxes
```

- `offers[1]["food"]`: ដំបូង **ទីតាំង** ក្នុង list បន្ទាប់មក **key** ក្នុង record។
- ប្រព័ន្ធព័ត៌មានស្ទើរតែទាំងអស់ មានទម្រង់បែបនេះ។

`for`: ធ្វើរឿងដដែល សម្រាប់គ្រប់ item

```
for offer in offers:
    print(offer["id"], offer["food"], offer["portions"], "portions")
```

- បន្ទាត់ដែលដកចូល run **ម្តង សម្រាប់ item នីមួយៗ**។
- `offer` ចង្អុលទៅ item បច្ចុប្បន្ន។
- ទម្លាប់: ឈ្មោះ **ពហុវចន:** សម្រាប់ list ឈ្មោះ **ឯកវចន:** សម្រាប់ item មួយ។

accumulator: ចាប់ពីសូន្យ បូកម្តងមួយៗ

```
prices_riel = [4000, 1500, 2000]
total_riel = 0                      # 1. ចាប់ផ្តើមមុន loop
for price_riel in prices_riel:
    total_riel += price_riel        # 2. បូក item នីមួយៗ
print(total_riel)                   # 3. ប្រើវាក្រោយ loop
```

- `+=` មានន័យថា "បូកបន្ថែមលើអ្វីដែលមានរួចហើយ"។
- លំនាំដដែលនេះ ក៏ **រាប់** បានដែរ: `count = 0` រួច `count += 1`។

🤔 ចុះបើ `total_riel = 0` នៅ **ក្នុង** loop វិញ?

ស្វែងរក record តាម ID របស់វា

```python
def find_item(items, wanted_code):
    for item in items:
        if item["code"] == wanted_code:
            return item          # រកឃើញ: ឈប់ ហើយបញ្ជូនវាត្រឡប់ទៅវិញ
    return None                  # បានពិនិត្យគ្រប់ item: រកមិនឃើញ
```

- `return` បញ្ឈប់ loop ភ្លាមៗ។
- `return None` តម្រង់ជួរជាមួយ `for`: អ្នកអាចនិយាយថា "រកមិនឃើញ" បាន តែ **ក្រោយ** ពេលមើលគ្រប់យ៉ាងរួច។

⚖️ ល្អ ឬមានហានិភ័យ?

Version A (ពីជំនួយការ AI)

```
offers_by_food = {}
offers_by_food["Bread rolls"] = {
    "donor": "Sunrise Bakery",
    "portions": 30}
offers_by_food["Bananas"] = {
    "donor": "Green Market Stall",
    "portions": 40}
stored_total = 70
# ក្រោយមក ហាងនំទីពីរ:
offers_by_food["Bread rolls"] = {
    "donor": "Golden Oven Bakery",
    "portions": 12}
stored_total += 12
```

Version B

```
offer_records = []
offer_records.append({"id": "F001",
    "donor": "Sunrise Bakery",
    "portions": 30})
offer_records.append({"id": "F002",
    "donor": "Green Market Stall",
    "portions": 40})
offer_records.append({"id": "F003",
    "donor": "Golden Oven Bakery",
    "portions": 12})
total = 0
for record in offer_records:
    total += record["portions"]
```

គោលការណ៍ទី 3 · record មួយ សម្រាប់របស់ពិតមួយ

- **record មួយ** សម្រាប់សំណើពិតនីមួយៗ ដែលមាន **ID មួយមិនស្ទួន**។
- ជ្រើសរើស key ដែលមិនដែលស្ទួន: ID មិនមែនឈ្មោះអាហារ។
- **កុំរក្សាទុកព័ត៌មានដដែលពីរដង**: គណនាផលបូកពី record ទាំងឡាយ។

> ពេលអ្នកពិនិត្យ code របស់ AI ចូរសួរ: *តើមានអ្វីកើតឡើង ពេលរបស់ពីរមានឈ្មោះដូចគ្នា?*

អ្នកបង្កើត: ផលបូក និងការស្វែងរក

```
def total_portions(records):
    # ចាប់ពី 0 បូក "portions" របស់ record នីមួយៗ រួច return total
    return None

def find_offer(records, offer_id):
    # return record ដែលត្រូវគ្នា; ក្រោយ loop ចូរ return None
    return "?"

bridge.check("total_portions", total_portions)  # 100 ហើយ [] → 0
bridge.check("find_offer", find_offer)       # រកឃើញ F002, F999 → None
```

ភ្ជាប់ចូល: ប្រៀបធៀបជាមួយប្រព័ន្ធពិត

```
my_system = bridge.System(MY_SETTINGS)
my_system.run_day(show=False)
system_records = my_system.records()

print(total_portions(system_records))              # របស់អ្នក
print(my_system.numbers()["portions_offered"])    # របស់ប្រព័ន្ធ
print(find_offer(system_records, "F002"))
```

- វិធីពីរដែលឯករាជ្យពីគ្នា ចម្លើយតែមួយ: ទុកចិត្តទាំងពីរបានច្រើនជាងមុនបន្តិច។

🧭 ការសម្រេចចិត្តរបស់អ្នក · ប្រភេទណាខ្លះ?

```
MY_SETTINGS["CATEGORIES"] = ["bread", "rice meals", "fruit",
                             "vegetables", "snacks", "soup"]
```

	default categories	+ "soup"
ទទួលយកនៅមាត់ទ្វារ	សំណើ 4	សំណើ 5
ចំណែកដែលបានផ្តល់	100	125
បដិសេធនៅមាត់ទ្វារ	5	4

- ស៊ុបមី (25 ចំណែក នៅសល់ 3 ម៉ោង) ទ្បឹងទៅ **កំពូល** នៃបញ្ជី។
- សរសេរប្រភេទជា **អក្សរតូច**: `"soup"` មិនមែន `"Soup"` ទេ។

កិច្ចការសប្តាហ៍នេះ

- ⭐ **មូលដ្ឋាន:** P1–P4, បញ្ហាចីក្នុង Version A, `total_portions` និង `find_offer` ជាប់ ✅ ទាំងអស់ និងត្រូវនឹងប្រព័ន្ធ, កំណត់ត្រាការសម្រេចចិត្តសម្រាប់ `CATEGORIES`។
- ⭐⭐ **បន្ថែម:** S1 រាប់តាមប្រភេទ; `portions_by_category` ជាប់ ✅។
- សប្តាហ៍នេះ គ្មានកិច្ចការស្រាវជ្រាវថ្មីទេ។ ថ្ងៃផុតកំណត់នៃកិច្ចការស្រាវជ្រាវទី 2 តាមការប្រកាសក្នុងថ្នាក់។

សប្តាហ៍ក្រោយ · Functions ដែលអាចទុកចិត្តបាន · Midterm

function របស់អ្នក ដំណើរការលើទិន្នន័យគំរូ។ តើវានឹងដំណើរការលើ **គ្រប់** input ទេ? អ្នកនឹងសរសេរ **test** ដែលបញ្ជាក់រឿងនោះ។ វគ្គទី 3 គឺជា **midterm** ដែលធ្វើម្ហាក់ឯង។